ISLAMIC UNIVERSITY OF TECHNOLOGY

SWE 4301

LECTURE 13

Topic: Interface Segregation Principle

November 23, 2024

PREPARED BY

Maliha Noushin Raida

Lecturer, Department of CSE

Contents

1 INTRODUCTION

The Interface Segregation Principle (ISP) deals with the disadvantages of bulky interfaces. Classes that have bulky interfaces are classes whose interfaces are not cohesive. In other words, the interfaces of the class can be broken up into groups of member functions. Each group serves a different set of clients. Thus some clients use one group of member functions, and other clients use the other groups.

The ISP acknowledges that there are objects that require non-cohesive interfaces; however it suggests that clients should not know about them as a single class. Instead, clients should know about abstract base classes that have cohesive interfaces. Some languages refer to these abstract base classes as “interfaces”, “protocols” or “signatures”.

2 INTERFACE POLLUTION

Consider a security system. In this system there are Door objects that can be locked and unlocked and that know whether they are open or closed.

```
public abstract Door {
  abstract void lock();
  abstract void unlock();
  abstract boolean isDoorOpen();
}
```

This class is abstract so that clients can use objects that conform to the Door interface, without having to depend upon particular implementations of Door.

Now consider that one such implementation. TimedDoor needs to sound an alarm when the door has been left open for too long. In order to do this the TimedDoor object communicates with another object called a Timer.

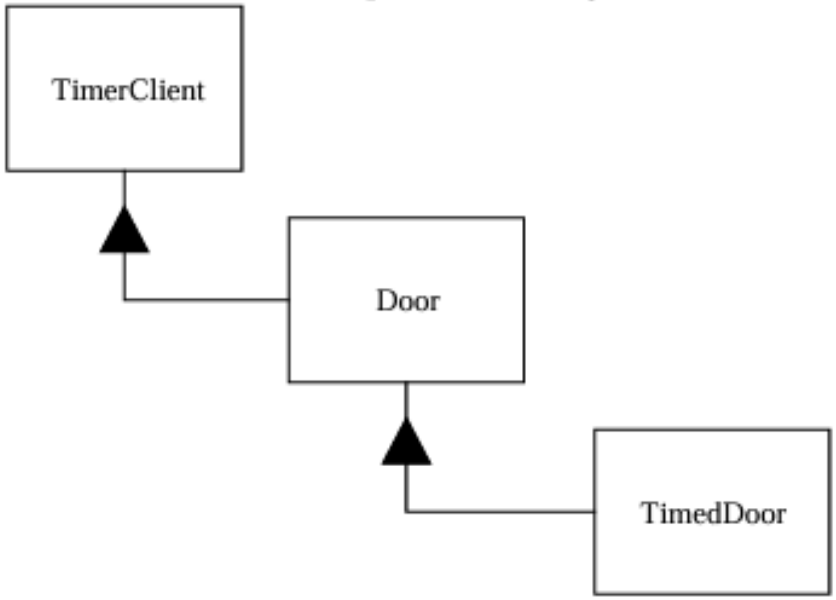
```
public class Timer {
    public void register(int timeout, TimerClient client) {
    // Implementation to register a timer with the given timeout and
   client
    }
}
public interface TimerClient {
    void timeOut();
}
```

When an object wishes to be informed about a timeout, it calls the Register function of the Timer. The arguments of this function are the time of the timeout, and a pointer to a

TimerClient object whose timeOut() will be called when the timeout expires.

How can we get the TimerClient class to communicate with the TimedDoor class so that the code in the TimedDoor can be notified of the timeout? There are several alternatives. Figure 1 shows a common solution. We force Door, and therefore TimedDoor, to inherit from TimerClient. This ensures that TimerClient can register itself with the Timer and receive the TimeOut message. The Door class now depends upon TimerClient. Not all varieties of

Figure 1
TimerClient at top of hierarchy



Door need timing. Indeed, the original Door abstraction had nothing whatever to do with timing. If timingfree derivatives of Door are created, those derivatives will have to provide nil implementations for the TimeOut method. This is the syndrome of interface pollution. The interface of Door has been polluted with an interface that it does not require. It has been forced to incorporate this interface solely for the benefit of one of its subclasses. If this practice is pursued, then every time a derivative needs a new interface, that interface will be added to the base class. This will further pollute the interface of the base class, making it "fat".

2.1 Separate Clients mean Separate Interfaces

Door and TimerClient represent interfaces that are used by complely different clients. Timer uses TimerClient, and classes that manipulate doors use Door. Since the clients are separate, the interfaces should remain separate too.

2.2 The backwards force applied by clients upon interfaces

We would be concerned about the changes to all the users of TimerClient, if the TimerClient interface changed. However, there is a force that operates in the other direction. That is, sometimes it is the user that forces a change to the interface.

For example, some users of Timer will register more than one timeout request. Consider the TimedDoor. When it detects that the Door has been opened, it sends the Register message to the Timer, requesting a timeout. However, before that timeout expires the door closes; remains closed for awhile, and then opens again. This causes us to register a new timeout request before the old one has expired. Finally, the first timeout request expires and the TimeOut function of the TimedDoor is invoked. And the Door alarms falsely.

We include a unique timeOutId code in each timeout registration, and repeat that code in the TimeOut call to the TimerClient. This allows each derivative of TimerClient to know which timeout request is being responded to.

```
1 public class Timer {
2     public void register(int timeout, int timeOutId, TimerClient client
    ) {
3         // Implementation to register a timer with the given timeout
    and client
4     }
5 }
6
7 public interface TimerClient {
8     void timeOut(int timeOutId);
9 }
```

Clearly this change will affect all the users of TimerClient. We accept this since the lack of the timeOutId is an oversight that needs correction. However, the design in Figure 1 will also cause Door, and all clients of Door to be affected by this fix. Why should a bug in TimerClient have any affect on clients of Door derivatives that do not require timing?

3 THE INTERFACE SEGREGATION PRINCIPLE

CLIENTS SHOULD NOT BE FORCED TO DEPEND UPON INTERFACES THAT THEY DO NOT USE.

When clients are forced to depend upon interfaces that they dont use, then those clients are subject to changes to those interfaces. This results in an unintentional coupling between all

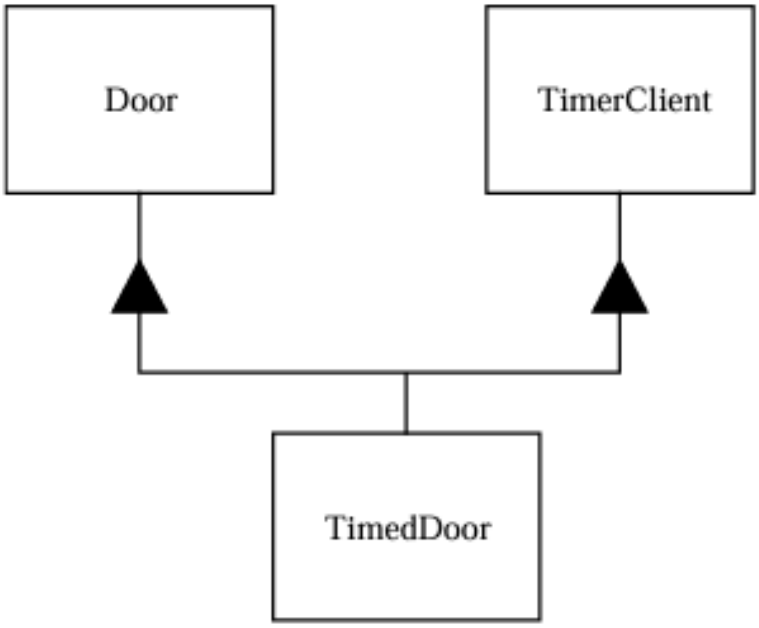
the clients. when a client depends upon a class that contains interfaces that the client does not use, but that other clients do use, then that client will be affected by the changes that those other clients force upon the class.

3.1 Implementation of ISP Using Delegation

```
// Base Door interface
public interface Door {
    void lock();
    void unlock();
    boolean isDoorOpen();
}

// Timer-related interface
public interface TimerClient {
    void timeout();
}

// Timer class remains the same
public class Timer {
    public void register(int timeout, TimerClient client) {
        // Implementation to register a timer with the given timeout
   and client
    }
}

// Separate class to handle timing behavior
public class DoorTimerAdapter implements TimerClient {
    private final Door door;

    public DoorTimerAdapter(Door door) {
        this.door = door;
    }

    @Override
    public void timeout() {
        door.lock();
    }
}

// TimedDoor delegates timer functionality to DoorTimerAdapter
```

```java
public class TimedDoor implements Door {
    private Timer timer;
    private DoorTimerAdapter timerAdapter;
    private boolean isLocked;
    public TimedDoor(Timer timer) {
        this.timer = timer;
        this.timerAdapter = new DoorTimerAdapter(this);
    }
    @Override
    public void lock() {
       isLocked = true;
    }
    @Override
    public void unlock() {
        isLocked = false;
        timer.register(300, timerAdapter);  // Example timeout of 300 seconds
    }
    @Override
    public boolean isDoorOpen() {
        return !isLocked ;
    }
}

// Concrete implementation of the Door interface
public class SimpleDoor implements Door {
    private boolean isLocked;
    @Override
    public void lock() {
        isLocked = true;
    }
    @Override
    public void unlock() {
        isLocked = false;
    }
    @Override
    public boolean isDoorOpen() {
        return !isLocked;
    }
}
```

3.2 Implementation of ISP Using Multiple Inheritance




```java
public class TimedDoor implements Door, TimerClient {
    private Timer timer;
    private boolean isLocked;
    public TimedDoor(Timer timer) {
            this.timer = timer;
    }
    @Override
    public void lock() {
       isLocked=true;
    }
    @Override
    public void unlock() {
         isLocked=false;
        startTimer();
    }
    @Override
    public boolean isDoorOpen() {
        return !isLocked;
    }
    private void startTimer() {
        timer.register(300, this);  // Example timeout of 300 seconds
    }
```

```
23      @Override
24      public void timeOut() {
25          lock();
26      }
27  }
```

3.3 Problems with large interface

- Big interfaces reduces cohesion
- Codesmell refused bequest